

Efecte laterale

1 Considerente teoretice

În lucrarea de față vom învăța despre efecte laterale care ne permit să manipulăm dinamic baza de predicate (baza de cunoștințe).

1.1 Efecte laterale

Efectele laterale se referă la manipularea dinamică a definițiilor de predicate și acest lucru se poate realiza cu următoarele predicate predefinite:

Predicat	Funcționalitate
<i>assert/1</i> (= <i>assertz/1</i>)	adaugă la sfârșitul bazei de cunoștințe clauza dată în argument
<i>asserta/1</i>	adaugă la începutul bazei de cunoștințe clauza dată în argument
<i>retract/1</i>	Încearcă unificarea și ștergerea primei clauze (sau fapt) cu care se poate unifica din baza de cunoștințe. Faptul/clauza unificată este ștersă din baza de cunoștințe
<i>retractall/1</i>	șterge toate clauzele care se unifică cu argumentul (în SWI-Prolog va merge la succes chiar dacă nu șterge nimic)

Predicatele care sunt definite și/sau apelate în fișierul „*pl*” se numesc predicate statice. Predicatele care sunt manipulate cu *assert/retract* se numesc predicate dinamice. În contrast cu predicatele statice pe care le-am văzut până acum, predicatele dinamice trebuie să fie declarate ca fiind dinamice.

În Prolog, un predicat este static sau dinamic. Un predicat static are faptele/regulile predefinite la începutul execuției și acestea nu se schimbă pe parcursul execuției. În mod normal, faptele/regulile sunt scrise într-un fișier de cod Prolog care este încărcat în timpul sesiunii Prolog. Dacă vrem să adăugăm faptele adiționale (sau chiar reguli) unui predicat, în timpul execuției unei întrebări, vom folosi "*assert/asserta/assertz*" sau în cazul în care vrem să ștergem fapte vom folosi "*retract/retractall*". Pentru a face acestea, predicatul trebuie să fie declarat dinamic.

Dacă un predicat este adăugat pentru prima dată cu *assert* atunci este implicit un predicat dinamic. Dacă vrem să manipulăm un predicat care apare în fișierul „*pl*” atunci trebuie să-l setăm ca și predicat dinamic adăugând următoare linie la începutul fișierului:

```
:-dynamic <nume_predicat>/<aritate>.
```

Când folosim aceste predicate trebuie să ținem cont de următoarele aspecte:

- **Backtracking-ul nu invalidează efectul la *assert***, ex: dacă un predicat a fost adăugat cu *assert*, rămâne în baza de predicate până când este șters explicit cu *retractm* chiar dacă nodul corespondent celui apel de *assert* este șters din arborele de execuție (ex: din cauza backtracking-ului).
- Predicatul *assert* întotdeauna rezultă în succes; nu face backtracking.

- Predicatul *retract* poate să eșueze, caz în care va porni **backtracking-ul**. În cazul predicatului *retract*, backtracking-ul invalidează temporar ștergerea pentru predicatele din același corp al clauzei cu apelul predicatului *retract* și care au fost apelate înaintea predicatului *retract*; astfel se păstrează *logical update view*.

Pentru a înțelege mai bine cum funcționează *retract* urmăriți execuția la:

Query

```
?- assert(insect(ant)), assert(insect(bee)), retract(insect(A)), writeln(A),
retract(insect(B)), fail.
```

Ați observat probabil că această întrebare ne dă rezultatul:

Answer

```
ant
bee
false
```

Deși al doilea *retract* șterge faptul *insect(bee)*, când se face backtracking și se ajunge la primul apel de *retract*, clauza este încă prezentă în logical view- practic, nu vede faptul că clauza a fost ștersă de al doilea apel de retract. Astfel *insect(A)* încă se poate unifica cu „bee”.

Prolog are și predicatul **retractall/1**, cu următorul comportament: va șterge toate clauzele predicatelor care se potrivesc cu argumentul. În unele versiuni de Prolog, retractall poate da fail dacă nu există nimic de extras. Pentru a evita acest lucru, ați putea să alegeți să faceți un assert a unei clauze dummy de tipul potrivit. Însă, în SWI Prolog, retractall reușește chiar și pentru un apel cu niciun fapt/regulă care să se potrivească.

Manipularea dinamică a bazei de cunoștințe prin assert/retract este folosită în special pentru salvarea rezultatelor de la calcule (memorisation/caching), astfel încât să nu fie pierdute prin backtracking. Astfel, dacă aceeași întrebare este pusă în viitor, răspunsul poate fi obținut fără a fi nevoie de a recalcula. Această tehnică se numește memorisation, sau caching, în unele aplicații poate să crească eficiența. De asemenea, aceste operații pot fi folosite pentru a schimba dinamic comportamentul unor predicate la rulare (meta-programming). Acest proces duce la un cod greu de urmărit. În cazurile cu mult backtracking, devine și mai greu. Prin urmare, această caracteristică non-declarativă a Prolog-ului ar trebui folosită cu atenție.

1.2 Fibonacci

1.2.1 Memorizare

Un exemplu de predicat care memorizează rezultatele parțiale, folosind efecte laterale, pentru a calcula al n-lea număr din *secvența Fibonacci* (ați întâlnit varianta mai puțin eficientă în laboratorul al doilea):

Code

```
:-dynamic memo_fib/2.

fib(N,F):- memo_fib(N,F), !.
```

```

fib(N,F):-
    N>1,
    N1 is N-1,
    N2 is N-2,
    fib(N1,F1),
    fib(N2,F2),
    F is F1+F2,
    assertz(memo_fib(N,F)).
fib(0,1).
fib(1,1).

```

Urmărește execuția la:

```

Query
?- fib(4,F), listing(memo_fib/2).
?- fib(10,F), listing(memo_fib/2).
% listing/1 - afișează toate clauzele predicatului memo_fib cu 2 argumente

```

1.2.2 Afișarea rezultatelor memorizate

Oricând avem nevoie să colectăm toate răspunsurile stocate în baza de predicate prin assert-uri, putem folosi **failure driven loops** care forțează Prolog-ul să facă backtracking până când rămâne fără posibilități. Tiparul pentru un failure driven loop care citește toate clauzele stocate – de exemplu pentru predicatul *memo_fib/2* de mai sus – și printează toate numerele *fibonacci* deja calculate:

```

Code
print_memo_fib:-
    memo_fib(N,F),
    format('memo_fib(~w, ~w).', [N, F]), nl,
    fail.
print_memo_fib.

```

Ne vom folosi de tehnica backtracking (forțând *fail*) pentru a parcurge toate clauzele predicatului *memo_fib/2* adăugate în baza de cunoștințe cu *assert*. **Deoarece se utilizează backtracking, faptele *memo_fib/2* nu se repetă.**

Urmărește execuția la:

```

Query
?- fib(4,F), print_memo_fib.
?- fib(10,F), print_memo_fib.

```

1.2.3 Colectarea rezultatelor memorizate

Pentru colectarea rezultatelor într-o listă ne putem folosi de un predicat predefinit: *findall*.

Urmărește execuția la:

Query

```
?- findall(X, append(X,_,[1,2,3,4]), List).  
?- findall(lists(X,Y), append(X,Y,[1,2,3,4]), List).  
?- findall(X, member(X,[1,2,3]), List).
```

Vom vedea un exemplu de folosire a efectelor laterale pentru a găsi toate răspunsurile unei întrebări: vom scrie predicatul care generează toate permutările unei liste și le returnează într-o listă. Vrem ca predicatul să funcționeze în felul următor:

Query

```
?- trace, all_perm([1,2,3],L).  
L=[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]];  
no.
```

Știind că deja puteți implementa predicatul *perm/2* – predicatul care generează o permutare a listei de intrare (vedeți documentul despre Metode de Sortare dacă nu) – specificația predicatului *all_perm/2* este:

Code

```
all_perm(L,_):-  
    perm(L,L1),  
    assertz(perm(L1)),  
    fail.  
all_perm(_,R):- collect_perms(R).  
  
collect_perms([L1|R]):- retract(perm(L1)), !, collect_perms(R).  
collect_perms([]).  
  
perm(L, [H|R]):- append(A, [H|T], L), append(A, T, L1), perm(L1, R).  
perm([], []).
```

Urmărește execuția la:

Query

```
?- retractall(perm(_)), all_perm([1,2],R), listing(perm/1).  
?- retractall(perm(_)), all_perm([1,2,3],R), listing(perm/1).
```

Întrebări:

1. De ce am nevoie de **retractall** înainte de apelul la **all_perm/2**?
 2. De ce este nevoie de **!** după **retract** în predicatul **collect_perms/1**?
 3. Ce tip de recursivitate este folosit în predicatul **collect_perms/1**? Se poate face colectarea în celălalt tip de recursivitate? Care este ordinea permutărilor în acest caz?
1. Predicatul **collect_perms/1** distruge rezultatele salvate?

1.2.3.1 De ce retractall?

Motivul utilizării predicatului **retractall** este de a ne asigura că baza noastră de cunoștințe este curată. Luați în considerare următorul predicat simplu care stochează valori între valorile *Low* și *High*. Apelurile secvențiale cu intervale suprapuse vor crea duplicate. Prin utilizarea predicatului **retractall**, ne putem asigura că lucrăm de la zero.

Code

```
store_nr_btw(Low, High):-  
    Low<High,  
    assertz(nr(Low)),  
    Low1 is Low+1,  
    store_nr_btw(Low1, High).  
store_nr_btw(High, High).
```

Urmărește execuția la:

Query

```
?- store_nr_btw(0, 2), listing(nr/1), store_nr_btw(0, 3), listing(nr/1).  
?- store_nr_btw(0, 2), listing(nr/1), retract_all(nr(_)), store_nr_btw(0, 3),  
listing(nr/1).
```

1.2.4 Procesul cu două stagii/doi pași al colectării din failure driven loop

Colectarea rezultatelor din buclele bazate pe eșec, cum este cazul predicatului *all_perm/2*, poate fi văzută ca un proces în două etape. Acestea includ un wrapper (ambalaj), *all_preds/2* în exemplul nostru, care are două clauze. Prima parte constă în procesarea și stocarea rezultatelor (prin utilizarea predicatului *assert*), realizată de prima clauză. Această primă clauză trebuie să eșueze (*fail*), astfel încât execuția să ajungă la a doua clauză, unde începe colectarea rezultatelor parțiale (prin utilizarea predicatului *retract*) iar rezultatul este construit într-o manieră backwards.

Code

```
all_preds(L,_):- store(L).  
all_preds(_,R):- collect(R).  
  
store(L):-  
    process(L, L1),          % process L to get L1  
    assertz(p(L1)),          % store L1  
    fail.  
  
collect([L1|R]):- retract(p(L1)), !, collect(R).  
collect([]).
```

1.3 Failure driven loop vs recursion

Am dori să evidențiem o comparație între buclele bazate pe eșec și predicatul recursiv, și care dintre ele este mai potrivit în diferite scenarii. Principala problemă a recursivității în cazul citirii bazei de date (memoriei) este că fiecare apel recursiv va citi același fapt, ceea ce duce la un ciclu infinit. Totuși, datorită mecanismului de backtracking inherent al lui `fail`, în buclele bazate pe eșec se poate citi cu ușurință baza de date.

O modalitate de a implementa o astfel de buclă folosind recursivitate, care să citească baza de date, este prin utilizarea predicatului *retract* (aceeași abordare poate fi aplicată și în cazul buclelor bazate pe eșec, însă acolo nu este necesară). Aceasta garantează că nu apare o buclă infinită, deoarece inputul este eliminat. Totuși, acesta este un **dezavantaj, deoarece distruge datele de intrare**. Așadar, pentru a păstra datele de intrare în cazul folosirii recursivității, trebuie utilizat un artificiu. Trebuie creat un predicat suplimentar care să țină evidența celor deja accesate (în acest caz, predicatul *seen/1*). Totuși, această abordare este inefficientă: la fiecare pas recursiv, toate predicatele deja accesate sunt verificate din nou până când se găsește unul neaccesat.

Code	
<pre>:-dynamic p/1. p(1). p(2). p(3). p(4). p(5).</pre>	
<pre>% nu necesită retract failure_driven_loop1:- p(X), assert(q(X)), fail. failure_driven_loop1:- listing(p/1), listing(q/1).</pre>	<pre>% dar se poate implementa cu failure_driven_loop2:- retract(p(X)), assert(q(X)), fail. failure_driven_loop2:- listing(p/1), listing(q/1).</pre>
<pre>% retract este obligatoriu, % altfel buclă infinită recursion1:- retract(p(X)), assert(q(X)), recursion1. recursion1:- listing(p/1), listing(q/1).</pre>	<pre>% pentru a păstra p/1 % în baza de predicate, % necesită un predicat adițional, % este foarte inefficient recursion2:- p(X), not(seen(X)),!, assert(seen(X)), assert(q(X)), recursion2. recursion2:-</pre>

	listing(p/1), listing(q/1), listing(seen/1). % se poate să pară non-sensic, % este un caz simplu, dar % când q/1 este un proces, % este necesar
--	--

Urmărește execuția la:

Query
?- trace, failure_driven_loop1. ?- trace, failure_driven_loop2. ?- trace, recursion1. ?- trace, recursion2.

1.4 Predicatul Univ

Cunoscut și sub numele de infix:

$X = ..L$

Acest predicat reprezintă operația în care lista L conține ca prim element functorul lui X, urmat de argumentele lui X, și are două funcționalități:

- **if** var(X) **then**
 - L este folosită pentru a construi structura corespunzătoare a lui X; capul listei L trebuie să fie un atom și va deveni functorul lui X.
 - **if** nonvar(X) **then**
2. functorul lui X trebuie să se potrivească cu primul element (capul) listei L, iar argumentele lui X trebuie să se potrivească cu coada listei L (argumentele lui X de la stânga la dreapta corespund elementelor cozii listei L de la stânga la dreapta).

Query
?- X = ..[a,b,c,d]. X = a(b,c,d)
?- X = ..[member,a,[b,c]]. X = member(a,[b,c]).
?- f(a,b,c) = ..X. X = [f,a,b,c].
?- append([H T],L,[H R]) = ..X. X = [append,[H T],L,[H R]].

Prin intermediul predicatului univ/infix, putem crea predicate mai generale care permit aplicarea regulilor după propria noastră alegere. De exemplu, dorim să creăm un predicat care ne permite să aplicăm o funcție (în Prolog, o regulă) tuturor elementelor sale, numit *map/3*.

```
Code
map(_, [], []).
map(Pred, [H|T], [H1|R]) :-
    P=..[Pred, H, H1],    % crearea unei predicat cu univ → Pred(H, H1)
    call(P), !,           % apelarea predicatului creat
    map(Pred, T, R).

double(X, Y) :- Y is X * 2.
halve(X, Y) :- Y is X / 2.
% ... se poate adăuga orice funcție dorită
```

1.4.1 Problema cu univ și SWISH

Combinăția de univ și call creează dinamic un predicat care este apelat, ceea ce nu este permis în SWISH, ducând la o eroare de „Sandbox restriction”.

1.4.2 Soluția: Artificiu

Putem folosi predicatul *call/n* care are comportamentul lui univ și call simultan, creează un apel de predicat cu primul argument ca și predicat, iar restul devin argumentele acestui predicat și apelează.

```
Code
map1(_, [], []).
map1(Pred, [H|T], [H1|R]) :-
    call(Pred, H, H1), !,    % se comportă ca și univ → Pred(H, H1)
    map1(Pred, T, R).

double(X, Y) :- Y is X * 2.
halve(X, Y) :- Y is X / 2.
% ... se poate adăuga orice funcție dorită
```

Follow the execution of:

```
Query
?- trace, map1(double, [1, 2, 3], Result).
Result = [2, 4, 6].

?- trace, map1(halve, [2, 4, 6], Result).
Result = [1, 2, 3].
```

2 Exerciții

Înainte de începe exercițiile, adăugați următoarele fapte care definesc arbori (complet și incomplet binar) în script-ul Prolog:

```
Code
% Trees:
incomplete_tree(t(7, t(5, t(3, _, _), t(6, _, _)), t(11, _, _))).
complete_tree(t(7, t(5, t(3, nil, nil), t(6, nil, nil)), t(11, nil, nil))).
```

Scrieți un predicat care:

3. Generează toate descompunerile posibile a unei liste în doua sub-liste fără a folosi predicatul predefinit *findall*.

```
Query
?- all_decompositions([1,2,3], List).
List=[ [[], [1,2,3]], [[1], [2,3]], [[1,2], [3]], [[1,2,3], []] ] ;
false
```

4. Calculează numărul de apariții al elementelor unice dintr-o listă și creează lista care conține perechi Element-Număr folosind *assert* și *retract*.

```
Query
?- count_elements([a,b,a,b,c], R).
R = [a-2, b-2, c-1]
```

5. Colectează (folosind retract-uri printr-un proces format din 2 pași) într-o **listă diferență** toate elementele pare ale unei liste **incomplete** date.

```
Query
?- even_dl([1,2,3|_], S, E).
S = [2|E]
```

6. Colectează (folosind retract-uri printr-un proces format din 2 pași) toate nodurile interne ale unui arbore binar **incomplet**.

```
Query
?- incomplete_tree(T), internal_list(T, R).
R = [7, 5|_].
```

7. Filtrează elemente bazat pe o funcție dată (*sugestie*: predicatul *univ*).

```
Query
?- filter(odd, [1, 2, 3, 4], Result).
Result = [1, 3].
```

8. Returnează true dacă oricare element satisface o funcție dată (*sugestie*: predicatul *univ*).

Query

```
?- any(greater_than_three, [1, 2, 4]).  
true.
```

9. Returnează true dacă toate elementele satisfac o funcție dată (*sugestie*: predicatul univ).

Query

```
?- all(positive, [1, 2, 3]).  
true.
```